

100 Days of Azure DevOps

Day 51 Handout — Ship the process with its layers — not a prayer that prod looks like your laptop

6 - Containers & Kubernetes · Personal lab only · Educational content · Not a sales pitch

Architecture A — image, container, layer, tag (same Dockerfile, four products)

Object	What it is	What people confuse
Image	Immutable filesystem layers + config, addressed by digest sha256:...	The container that happens to be running on the laptop
Container	A running (or stopped) instance of an image. Writable layer on top	docker commit of a mutated container as the new source of truth
Layer	Diff from one Dockerfile instruction. Cache key is that instruction + parents	Cache as a miracle rather than COPY/RUN order
Tag	Mutable pointer (myapp:day51) that can move to a new digest	:latest as a version. latest is a moving sign
Digest	Immutable content address. Pull by @sha256:... when you mean exactly this bits	Optional decoration. Promotion identity lives here, not on latest

Architecture B — decide the Dockerfile like a cache design, not a blog paste

Constraint	Default	Change when
Base	FROM node:20-alpine (pin major). Not node:latest	You have a reason for distroless/debian — write it
Dependency cache	COPY package*.json → RUN npm ci --omit=dev → COPY .	Never COPY . before install. README edits must not bust npm ci
Runtime fat	Single stage for a hello app	Build needs a compiler the runtime image must not ship — then multi-stage
Secrets	.dockerignore excludes .env, .git, node_modules	A secret in the image is in every layer forever
Proof	docker build + docker run on the personal PC	ACR is tomorrow. Local process must start today

Architecture C — pitfalls that turn a 8s rebuild into a 5-minute customs check

Pitfall	What you see	Fix
COPY . before npm ci	Every source typo rebuilds node_modules	Reorder: manifests, install, then source
FROM node:latest	Tuesday image SKU change, different Node	Pin 20-alpine; digest-pin if you are paranoid
No .dockerignore	Image contains .git, .env, local node_modules	Add .dockerignore before the next build
Mutate then commit	Snowflake container nobody can rebuild	Change the Dockerfile. Rebuild. Do not commit a running box
Tag-only identity	Which latest did we ship Friday?	Record the digest. Treat tags as pointers

One-liner to remember

Containers are shipping containers for processes — same app, fewer 'works on my laptop' customs checks. Layer cache is a design, not a miracle.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal PC and personal Azure DevOps project azure-100-labs only. No employer Docker Desktop, no client app.

- 1 Install Docker Desktop on the personal PC (or confirm docker version already works). Do not use an employer-licensed install for this series.
- 2 In the sample app repo, add a Dockerfile: FROM node:20-alpine, WORKDIR /app, COPY package*.json, RUN npm ci --omit=dev, COPY ., CMD. Add a .dockerignore that excludes .env, .git, node_modules.
- 3 docker build -t myapp:day51 . Note which steps show CACHED vs RUN. Touch README (or a source file) and rebuild. Touch package.json and rebuild. Write which layer busted.
- 4 docker run --rm -p 3000:3000 myapp:day51 Hit localhost until the process answers. docker image ls and docker inspect myapp:day51 --format '{{.Id}} {{.RepoTags}}'.
- 5 Write docs/docker-day51.md: image vs container in one sentence each, the cache observation, and the digest (or Image ID) you built. ACR is tomorrow — do not push yet.
- 6 docker image prune is optional. Do not docker commit. If the app is not Node, same order: lockfile copy, install, then source.

Starter Dockerfile — cache order is the product

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Done checklist

- Can explain image vs container vs tag vs digest without saying the container is the image
- Dockerfile copies lockfile/manifests before source; .dockerignore exists
- Noted cache bust: source change vs package.json change
- docs/docker-day51.md on the personal repo; no employer screenshot

Tomorrow — Day 52

Azure Container Registry — private closet for images